Pilotare il 74HC595: dalla teoria allo schema KiCad

Lezione 4 — 2MAK, SST-MEC-25c — Shift register in pratica

01/10/2026

1 Ripasso: cos'è il 74HC595

Nota

Il **flip-flop di tipo D** è un componente con due ingressi (**CLK**, **D**) e due uscite (Q, $\overline{Q}$). Ogni volta che il filo `CLK` riceve un **fronte di salita** (una transizione da basso ad alto), il flip-flop "**scatta una fotografia**" del livello presente in quel momento sul filo `D` e lo salva su Q: il filo `D` può variare liberamente nel tempo, ma Q **resta fermo** all'ultimo valore fotografato finché non arriva un nuovo fronte di salita. È quindi una **cella di memoria**.

Nota

Il **contatore asincrono**, già visto, è una **batteria di flip-flop D** collegati in cascata: parte da **tutti zero** e, a ogni fronte di salita, **incrementa di uno** il numero binario rappresentato dalle uscite. Con 4 flip-flop si arriva fino a `1111` (il 15 decimale), poi il conteggio **torna a zero** – per rappresentare numeri più grandi servono **più flip-flop** in cascata. È **asincrono** perché il clock non arriva direttamente a tutti i flip-flop insieme: solo il primo riceve il **clock** "vero", mentre ogni stadio successivo è pilotato dall'**uscita dello stadio precedente**.

Nota

Ogni stadio successivo del contatore **dimezza** la **frequenza di cambiamento** rispetto allo stadio precedente: il primo bit cambia a ogni fronte di clock, il secondo **ogni due fronti**, il terzo **ogni quattro**, e così via. Questo dimezzamento è legato alla relazione tra **frequenza e periodo**:

$$f = \frac{1}{T}$$

Se il **periodo** (il tempo che intercorre tra un evento e il successivo) raddoppia, la **frequenza** (il numero di eventi al secondo) si **dimezza** – ed è esattamente quello che succede **stadio dopo stadio** in un contatore asincrono.

Definizione

Il **74HC595** ("**595** per gli amici") è uno **shift register** seriale-parallelo a **8 bit**. "Seriale-parallelo" significa che i dati **entrano** uno alla volta (**in serie**, un bit dopo l'altro, come infilare palline una alla volta in un tubo), ma possono essere **letti tutti insieme** in **parallelo** sugli 8 piedini di uscita.

Nota

La sigla del componente si legge così: `74` indica la **famiglia** (**uso commerciale**, a differenza della famiglia `54` per uso militare, con range di temperatura più ampio); le **due lettere** nel mezzo (`HC`, `LS`, `HCT`, ...) indicano la **tecnologia di costruzione** del circuito interno – non un materiale fisico diverso, ma un modo diverso di realizzare internamente la stessa funzione logica. **`HC`** (High-speed CMOS) è la tecnologia più moderna e a **basso consumo**, oggi quasi sempre disponibile; **`LS`** è una tecnologia più vecchia, più veloce ma con **consumi maggiori**. Nello schema elettrico conta solo il **simbolo generico** `74HC595`: se poi il componente fisico reperito è un `74LS595`, va bene comunque, perché la **funzione logica** è identica – cambia solo la **tecnologia costruttiva**, un dettaglio che riguarda chi progetta applicazioni sofisticate, non il livello di questo corso.

Esempio

Un'analogia con la **stampa 3D**: una stessa stampante può realizzare lo **stesso identico oggetto** in **materiali diversi** – **PLA**, **PETG**, **TPU**, **ASA**. L'oggetto è lo stesso (stessa forma, stessa funzione sulla carta), ma il materiale ne cambia le **caratteristiche fisiche**: un martello stampato in PETG, se colpisce qualcosa, **regge l'urto**; lo stesso martello stampato in PLA rischia di **rompersi**, perché il PLA è più **fragile**. Allo stesso modo, un 74HC595 e un 74LS595 fanno "sulla carta" la stessa funzione logica, ma le due tecnologie costruttive hanno **caratteristiche elettriche** diverse: HC **consuma pochissimo**, LS è **più veloce** ma **consuma di più**. Per chi sta progettando un'applicazione sofisticata, questa differenza può essere determinante nella scelta del componente – per chi sta ancora imparando le basi dell'elettronica, come in questo corso, è un dettaglio a cui **non serve dare peso**: ci si limita a prendere la **variante** che si trova più facilmente in catalogo.

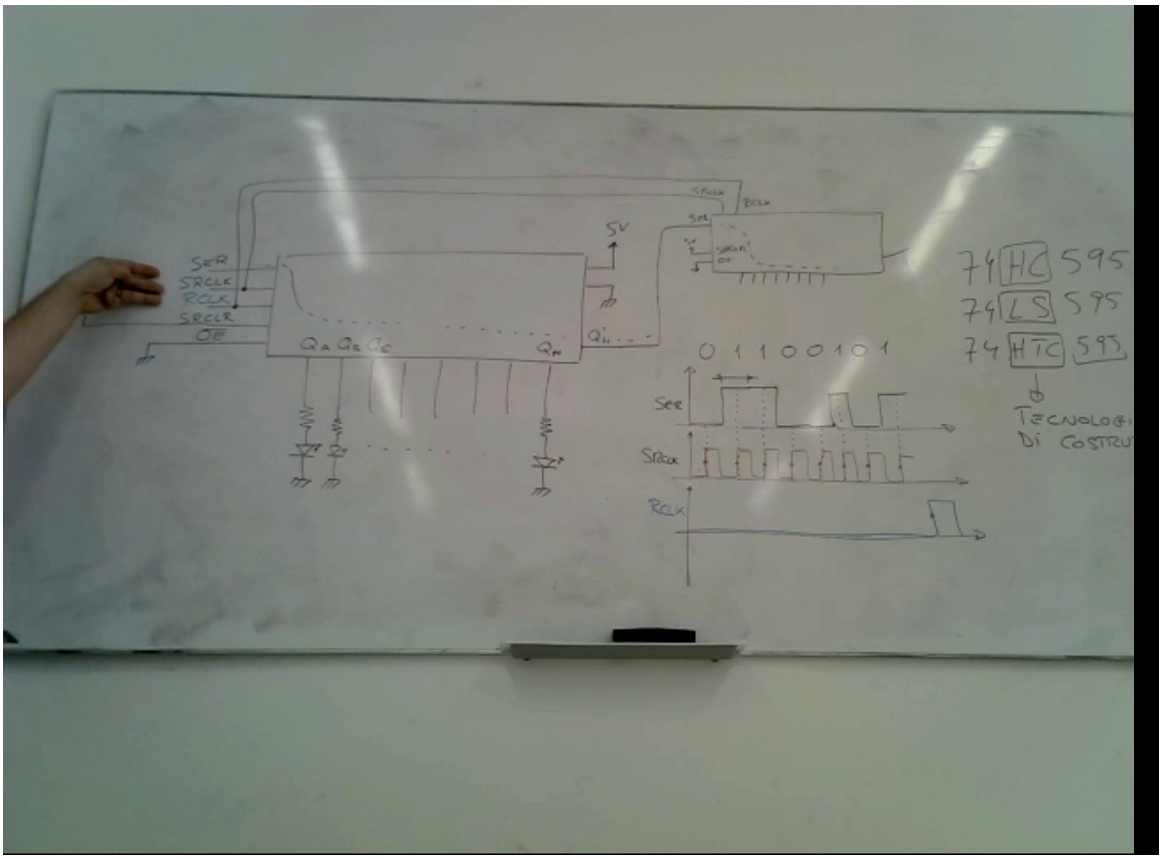


Figura 1: Il 74HC595 disegnato alla lavagna: a sinistra i cinque piedini di controllo (SER, SRCLK, RCLK, SRCLR, OE), a destra le otto uscite Q_A–Q_H con il piedino di cascata Q'_H verso un secondo 595; in basso il diagramma temporale di SER/SRCLK/RCLK per la sequenza di esempio `01100101`, e a destra le tre varianti di sigla (HC/LS/HTC) incontrate cercando il componente.

2 I cinque piedini di controllo

Definizione

Il 74HC595 ha **cinque piedini** dedicati al **pilotaggio dei dati**:

- **SER**: l'**ingresso dati seriale**, collegato direttamente all'ingresso D del **primo flip-flop** interno. È il filo su cui si presenta, **un bit alla volta**, la sequenza di 0 e 1 da inserire.
- **SRCLK**: il **clock dello shift register**. Ogni **fronte di salita** fa **avanzare di una posizione** il bit appena presentato su SER – esattamente come il clock di un flip-flop D.
- **RCLK** (*Register Clock*): **pubblica sulle uscite**, tutte insieme, gli 8 bit caricati nello **shift register interno**. Resta fermo durante il caricamento dei singoli bit, e riceve un **singolo fronte di salita** solo alla fine, dopo che tutti e 8 i bit sono stati inseriti.
- **SRCLR** (*Serial Clear*, negato): se portato a **livello basso**, **cancella tutte le uscite**. Va sempre tenuto **collegato** a $+5$ V (livello alto, cioè "non attivo"), perché non vogliamo che il registro venga **mai resettato** durante il normale funzionamento.

- **OE** (*Output Enable*, negato): **abilita le uscite**. Va sempre **collegato a GND** (livello basso, cioè "**sempre abilitato**").

Nel nostro uso, SRCLR e OE restano **fissi** (rispettivamente a $+5$ V e GND): i soli **tre piedini** che il microcontrollore deve effettivamente **pilotare** sono **SER**, **SRCLK** e **RCLK**.

Nota

Il piedino **OE** controlla una proprietà delle uscite chiamata **tristate** (o *three-state*): oltre ai normali **livelli 0 e 1**, un'uscita tristate può assumere un terzo stato, di **alta impedenza**, in cui risulta **fisicamente scollegata** dal resto del circuito. Questa funzione serve, ad esempio, quando più dispositivi devono **condividere lo stesso filo** (bus): **solo uno alla volta** "parla" sul filo, mentre tutti gli altri restano scollegati grazie al tristate. Nel nostro uso **non sfruttiamo** questa funzione, perché teniamo OE sempre a **livello basso** (uscite sempre abilitate).

Esempio

In classe, per fare un esempio concreto, è stato chiesto a un allievo di scegliere una **sequenza di 8 bit** a piacere: la scelta è caduta su `0 1 1 0 0 0 1 0`. Usando questa sequenza, la **procedura** per scrivere il **byte** sulle **8 uscite** del 595 è:

1. Si **presenta** il **primo bit** (`0`) su SER.
2. Si genera un fronte di salita su SRCLK: il bit **avanza** nello shift register interno.
3. Si **ripete** per ciascuno degli **8 bit** della sequenza, nell'ordine: `0, 1, 1, 0, 0, 0, 1, 0`.
4. Solo dopo aver inserito **tutti e 8 i bit**, si genera un singolo fronte di salita su **RCLK**: a questo punto, e **solo a questo punto**, il dato viene **pubblicato sulle uscite**.

Nota

Perché il bit va presentato a **metà** dell'**intervallo di tempo** dedicato a quel bit, e non subito all'inizio? Perché all'inizio il segnale è troppo vicino alla **transizione** del dato precedente, con il rischio di leggere un valore **ancora in fase di cambiamento**. È come saltare su uno scalino: non si salta sul bordo, ma **al centro**, per essere sicuri di centrarlo.

Esempio

Quanto deve durare, in pratica, ciascun bit? Dipende da quanto **velocemente** si vuole "riempire" il registro. Un **pannello con 8192 LED** (1024 byte) da aggiornare in un secondo richiede una **frequenza di SRCLK** molto più alta di un singolo 595: per un display che deve apparire fluido all'**occhio umano**, serve **rinfrescare l'immagine** ad almeno **20 Hz**, meglio ancora **50 Hz** – il che significa lavorare con tempi per bit nell'ordine dei **microsecondi o millisecondi**, non dei secondi.

Attenzione

Perché servono **due** registri distinti (lo **shift register interno** pilotato da **SRCLK**, e il **registro di uscita** pilotato da **RCLK**) invece di uno solo? Se le uscite si aggiornassero a **ogni singolo colpo** di SRCLK, si vedrebbe visivamente lo **scorrimento** del dato lungo gli 8 LED collegati, bit dopo bit, invece di un **cambiamento istantaneo**. È come **coprire il tubo** delle palline mentre lo si riempie, e scoprirlo (scattare la fotografia) solo quando tutte le palline sono al loro posto: **separando i due clock**, il cambiamento sulle uscite appare **istantaneo e pulito**, senza mostrare il passaggio intermedio.

3 Le otto uscite e il collegamento in cascata

Definizione

Le **otto uscite** del 74HC595 si chiamano $Q_A, Q_B, Q_C, Q_D, Q_E, Q_F, Q_G, Q_H$. A ciascuna si può collegare **qualunque carico** (un **LED** con la sua **resistenza di limitazione**, un transistor, ecc.): la cosa notevole è che per pilotare **8 uscite indipendenti** si occupano solamente **tre** fili del microcontrollore (SER, SRCLK, RCLK).

Esempio

Esiste un nono filo, chiamato Q'_H (o Q_H primo): è una **copia del segnale** che, internamente, uscirebbe dal nono flip-flop se la catena continuasse. Collegando Q'_H del primo 595 al piedino SER di un **secondo 595** (e **condividendo SRCLK e RCLK** tra tutti i chip collegati in cascata), si ottengono **16 uscite** pilotate sempre con soli tre fili del microcontrollore. Si possono mettere in **cascata** quanti 595 si vuole, senza un **limite fisico** teorico: un **pannello LED monocromatico** da 32×16 moduli (512 LED totali) funziona esattamente con questo principio, e i **cartelloni pubblicitari** o i grandi schermi usati ai concerti sono costruiti con la stessa logica, solo su scala molto più grande.

4 Disegnare il circuito in KiCad

Nota

Nel cercare il componente nella **libreria di KiCad** compaiono più varianti (`74HC595`, `74HCT595`, `74LS595`, ecc.): la scelta del componente logico **non è critica**, **qualunque variante** va bene per lo schema. Conviene però definire subito l'**impronta** (footprint) giusta, perché cambiando le opzioni di ricerca cambiano anche le impronte disponibili – meglio sceglierla **una volta sola** invece di doverla cercare di nuovo in seguito.

4.1 Lo zoccolo (socket)

Definizione

Uno **zoccolo** (*socket*) è un **supporto di plastica** con i piedini che vengono **saldati alla scheda**, e con delle fessure superiori in cui si incastra il componente vero e proprio, **senza saldarlo** direttamente. Permette di **smontare e rimontare** il componente **senza dissaldare**: utile se ci si accorge di averlo montato **al contrario** (basta sfilarlo e girarlo), o se si brucia (si sfila e se ne mette uno nuovo senza rifare la saldatura).

Esempio

Non è un rischio teorico: l'anno precedente, un allievo aveva effettivamente montato un componente al contrario sulla scheda, senza controllare la **tacca di riferimento** – se il componente fosse stato saldato direttamente, **senza zoccolo**, sarebbe stato necessario **dissaldarlo** e rimontarlo da capo, un lavoro lungo e delicato (**rischio concreto** di danneggiare le **piste** del circuito stampato nel procedimento).

Esempio

Un chip può costare anche **diversi euro** (es. 15 €): se il circuito va buttato via perché qualcosa non funziona, con lo zoccolo si perde **solo lo zoccolo** stesso (pochi centesimi), non il chip. Per chi sta ancora imparando – dove la probabilità di un **errore di montaggio** non è mai davvero zero – montare **sempre lo zoccolo** è **obbligatorio**, non facoltativo.

Nota

Esistono **due tipologie costruttive** di zoccolo: a **lamella**, dove una **linguetta metallica** piegata forma un "invito" in cui si infila il piedino del componente, che resta poi incastrato tra due lamelle; e a **tulipano**, dove il contatto è un piccolo **cilindro pieno** (tornito), con un foro al centro in cui entra il piedino – **più solido e preciso**, ma anche **più costoso**. Per **uso scolastico** si usa quasi sempre quello a lamella, anche se è leggermente più delicato e rischia di piegarsi con l'uso.

Nota

Esiste anche una terza categoria, lo zoccolo **ZIF** (*Zero Insertion Force*): funziona con lo stesso principio di montaggio degli zoccoli delle CPU nei computer desktop – una **leva** che si solleva per inserire il componente **senza forzarlo**, poi si abbassa per **bloccarlo** in posizione. **Non è utilizzato** per i nostri circuiti, pensato com'è per componenti ben più delicati e costosi di un 74HC595.

4.2 Scegliere i footprint corretti

Definizione

Per ogni componente, oltre al simbolo elettrico, va assegnata un'**impronta** (footprint) che descrive il **disegno fisico** sul PCB:

- **Resistore**: tipo `THT` (a **foro passante**, non SMD), con passo `P10,16` (10,16 mm tra i due terminali).
- **LED**: da **5 mm**, versione **senza cupola trasparente** (*flat top*).
- **Connettore verso il microcontrollore**: tipo `pin header`, **passo standard** 2,54 mm, orientamento **verticale** (per restare coerenti con gli **zoccoli di espansione** tipici di una scheda Arduino).
- **595**: contenitore `DIP`, con **socket** (zoccolo) anziché saldatura diretta.

Nota

Per **assegnare un'impronta**: **selezionare il componente**, premere il tasto `F` (footprint), aprire la libreria con l'icona dedicata, e usare i **due filtri** in alto ("filtra per piedino", "filtra per numero di pin") per **restringere rapidamente la ricerca** prima di scegliere.

Attenzione

Nella scelta del footprint dello zoccolo, tra piedini **long pad** (allungati) e **round pad** (tondi), conviene sempre preferire il **long pad** per **saldature fatte a mano** da chi sta ancora imparando: un **pad più piccolo** si scalda più in fretta durante la saldatura, con il rischio che la **colla** che tiene il rame attaccato al circuito stampato si sciolga e il **pad si stacchi**. Un **pad più grande** richiede un po' più stagno, ma concede **più tempo** per completare la saldatura senza rischiare il distacco, ed è anche **meccanicamente più solido**.

Esempio

Un episodio concreto, osservato durante il lavoro in classe: un allievo aveva scelto il **connettore sbagliato** per J1 (il connettore verso il microcontrollore), senza seguire l'istruzione già data in precedenza. La **procedura di correzione**, guidata passo-passo dal docente: sullo schema elettrico, selezionare il connettore, premere `F` per aprire la libreria, e spuntare le **due caselle** "applica filtri impronte" – una per filtrare per **numero di piedini**, l'altra per il **passo**. Scendendo nell'elenco fino al passo corretto (2,54 mm, verticale), si seleziona il **footprint giusto** e si importa. Dopo aver cambiato footprint, è necessario andare nel pannello "ports" per **verificare e spostare** tutte le **connessioni** già fatte, perché cambiando footprint può cambiare la **disposizione fisica dei piedini** da collegare.

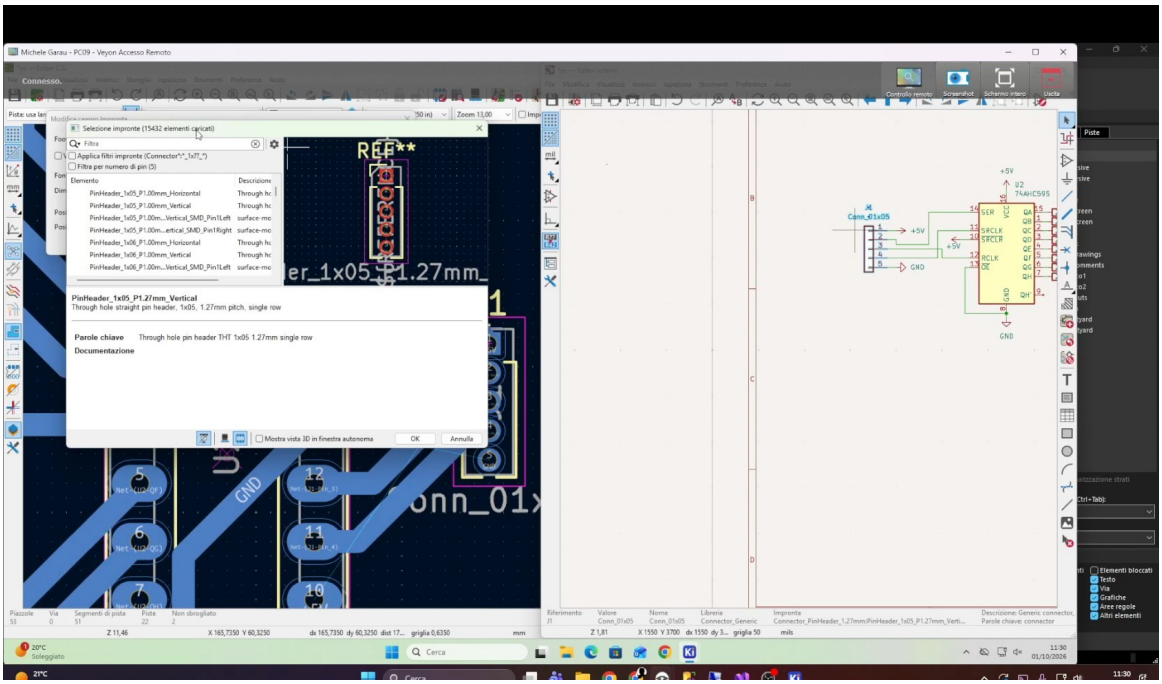


Figura 2: La finestra "Selezione impronte" di KiCad durante la ricerca del footprint del connettore J1: l'elenco iniziale mostra alcune varianti `PinHeader_1x05` con passo diverso da quello che serve a noi (1,00 mm, 1,27 mm) – bisogna scorrere l'elenco fino a trovare la variante verticale a passo **2,54 mm**, quella coerente con gli zoccoli di espansione di una scheda Arduino. A destra, lo schema elettrico del 595 con J1 già collegato ai cinque segnali.

4.3 Collegare l'alimentazione e disporre i componenti

Esempio

Alimentazione e massa si collegano con le etichette standard **+5V** e **GND**, prese dal selettore dei **simboli di potenza**. **Regola importante**: una volta scelto un **simbolo di GND**, va usato **sempre** lo **stesso** su tutto lo schema – **simboli diversi** per lo stesso riferimento vengono trattati da KiCad come fili elettricamente **staccati** tra loro, anche se visivamente sembrano la stessa cosa.

Nota

Per evitare di dover collegare un simbolo GND separato a ciascuno degli 8 LED, conviene disegnare un **unico filo** che li collega tutti insieme, con un **solo simbolo GND** finale.

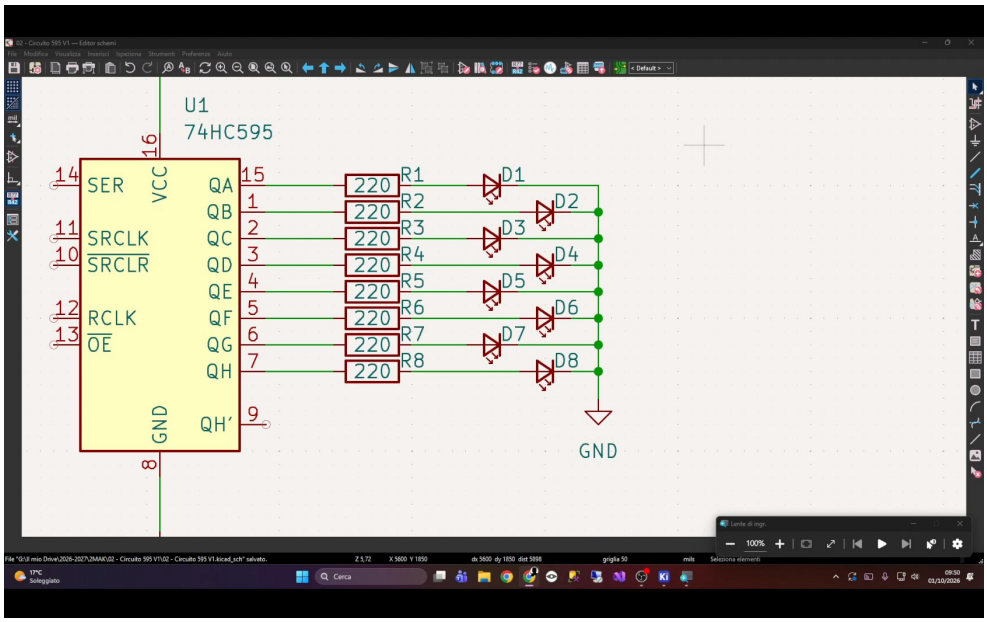


Figura 3: Schema elettrico completo degli otto LED con le rispettive resistenze da 220 Ω, collegati alle uscite Q_A–Q_H del 74HC595 (U1): un unico filo di massa comune collega tutti i catodi a un solo simbolo GND, invece di ripeterlo otto volte.

Esempio

Per **copiare un componente** già configurato (es. un resistore) su tutti gli altri senza che le etichette si sovrappongano visivamente: prima si **rifinisce completamente** il singolo componente (valore impostato, etichette spostate in una posizione pulita, footprint assegnato), e **solo dopo** lo si copia sugli altri. **Copiare prima e sistemare dopo** genera un **disordine grafico** inutile da correggere componente per componente.

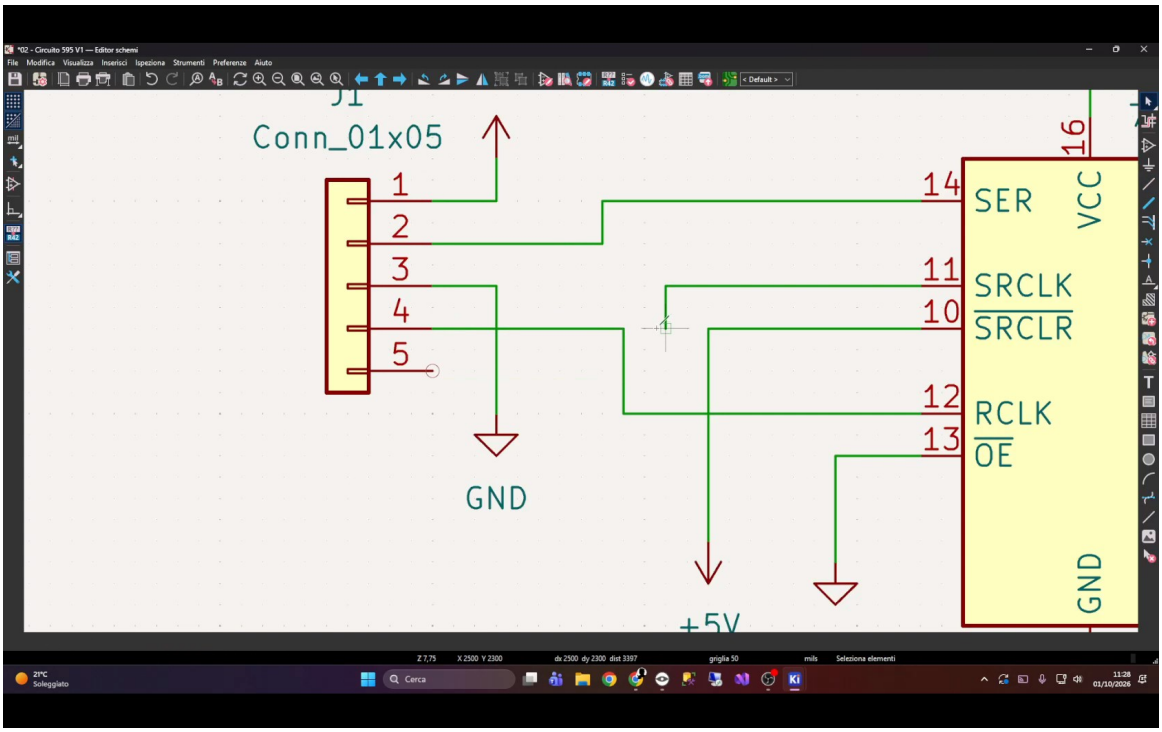


Figura 4: Il connettore J1 (verso il microcontrollore/Arduino) collegato ai cinque segnali del 595: piedino 1 a SER, 2 a SRCLK, 3 a GND, 4 a SRCLR (fissato a +5 V tramite il filo che scende dal pin 4), 5 lasciato libero; RCLK e OE fissati rispettivamente al piedino 12 e a GND.

Nota

Disposizione estetica consigliata: tenere tutti gli 8 LED **allineati e in ordine** (da Q_A a Q_H), invece che sparsi nella scheda – rende molto più facile leggere a colpo d'occhio il **dato binario** rappresentato. È una scelta legata allo **scopo del circuito**: se gli stessi LED dovessero far parte di un **pannello con posizioni vincolate** da un'altra esigenza (es. etichette da illuminare), la disposizione seguirebbe quel vincolo invece dell'ordine binario.

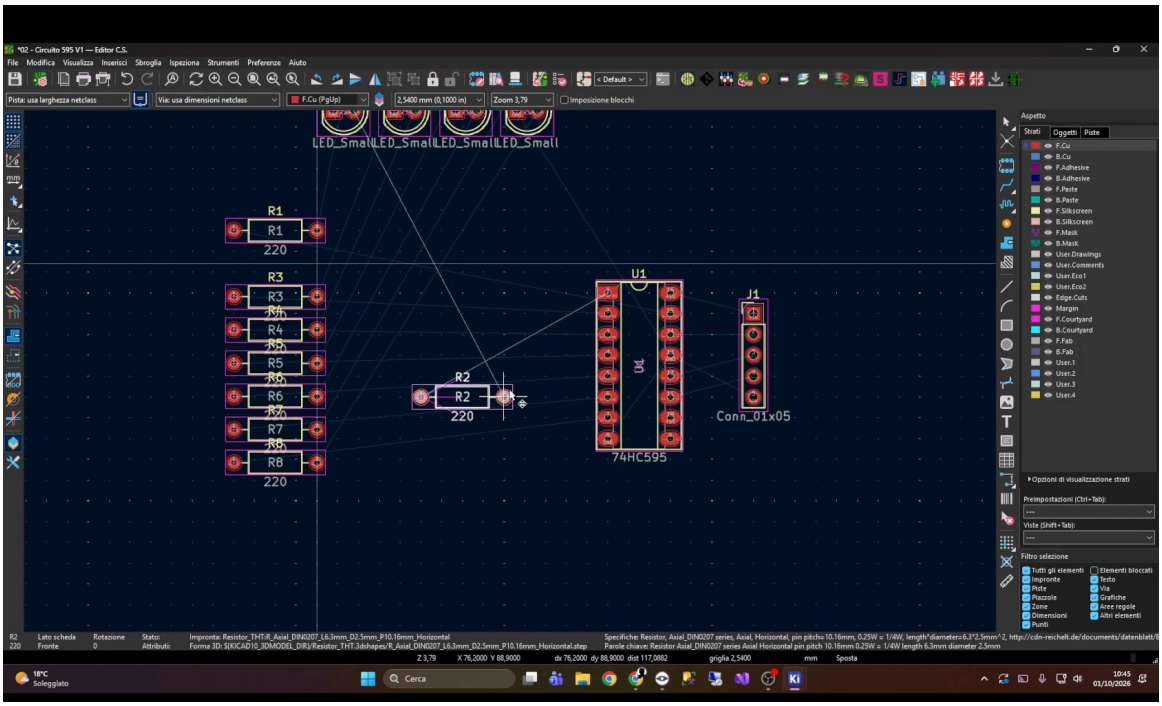


Figura 5: Il circuito stampato in fase di piazzamento: resistori (R1–R8) già allineati verticalmente a sinistra, il 595 (U1) e il connettore J1 (Conn_01x05) a destra, con le linee ratsnest (sottili, bianche) che mostrano ancora da tracciare le piste vere e proprie.

4.4 Larghezza delle piste e regole di sbroglio

Definizione

Una **net** è un insieme di punti del circuito elettricamente **equipotenziali** (collegati tra loro dallo stesso filo/pista): ogni net ha un proprio **identificativo**. La **net class** è l'impostazione (in KiCad: icona verde con l'ingranaggio → impostazioni scheda → net class) che permette di definire la **larghezza** delle piste per **gruppi di net**, oppure per tutte insieme (**classe di default**).

Nota

Il **limite costruttivo minimo** del nostro processo (**fotoincisione** fatta a mano) è circa **0,4 mm**, raggiungibile solo con **grande precisione**. Per un circuito semplice come questo, senza bisogno di **densità estrema**, conviene partire da **piste larghe 1 mm**: molto più del minimo tecnico, ma **molto più tollerante** a piccole **imprecisioni di produzione**.

Attenzione

Regola ferma per lo sbroglio: **mai tracciare** piste con **curve a 90 gradi**. Il verso corretto è **spostarsi leggermente fuori dal pad** e poi piegare, per tenere sempre una **distanza di sicurezza** tra piste vicine – se due piste restano troppo vicine, in fase di produzione (taglio manuale della scheda dal pannello) il rischio è dover **raschiare via il rame** in eccesso con un **taglierino**, con **margine di errore** pericolosamente basso.

Esempio

Due correzioni concrete osservate durante lo sbroglio in classe:

- Un allievo aveva tracciato **due piste** che partivano **troppo vicine** tra loro direttamente da due pad adiacenti. Correzione: quando si parte da due pad vicini, **non si tracciano mai** le piste dritte una accanto all'altra fin da subito – ci si **sposta leggermente fuori** da ciascun pad, e solo dopo si piega, lasciando libera la zona più stretta.
- Un altro allievo aveva realizzato, vicino al connettore J1 e a un filo di GND, una serie di **curve spezzate** (il filo di GND scendeva, scendeva ancora, poi risaliva) che allungavano inutilmente il percorso. Correzione: allineare il **piedino 3** di J1 con il piedino GND corrispondente permette una **pista dritta**, eliminando del tutto le curve superflue.

Per **correggere un tratto di pista** già tracciato senza dover rifare tutto da capo: selezionare il tratto e spostarlo con i tasti **freccia** della tastiera. Esiste anche la possibilità di spostare un intero componente con il tasto `G` (che **trascina con sé** anche le connessioni già fatte), ma va usato con cautela: non sempre funziona correttamente, e in alcuni casi può **disordinare le connessioni** invece di sistemarle.

Attenzione

Griglia di lavoro: non usare mai una **griglia in millimetri interi** (es. 5 mm) quando si spostano componenti con passo standard 2,54 mm o 1,27 mm – il componente finirebbe leggermente storto rispetto alla griglia dei piedini. Usare sempre una griglia a **2,54 mm o 1,27 mm** (i mils, l'unità di misura dei componenti elettronici, si passa ai millimetri solo per le misure meccaniche della scheda). Inoltre, usare la griglia **più grande possibile** che il passo dei componenti permette: una **griglia troppo fine** rende facile spostare un componente di una quantità minima senza accorgersene, mentre con una **griglia grossa** un **errore di posizionamento** si nota subito. Questa è stata, con un certo margine, la **regola più ripetuta** durante il lavoro pratico in classe – un segnale che vale la pena **controllarla sempre due volte** prima di spostare un componente.

Esempio

Criterio pratico per **minimizzare la lunghezza delle piste**: se un componente (es. l'integrato U1) è posizionato più lontano del necessario dal punto a cui deve collegarsi, va **spostato più vicino**. **Allineare i piedini** di due componenti collegati direttamente (es. un piedino del connettore J1 con il piedino di GND corrispondente) permette **piste dritte**, al posto di **curve spezzate** che allungano inutilmente il percorso.

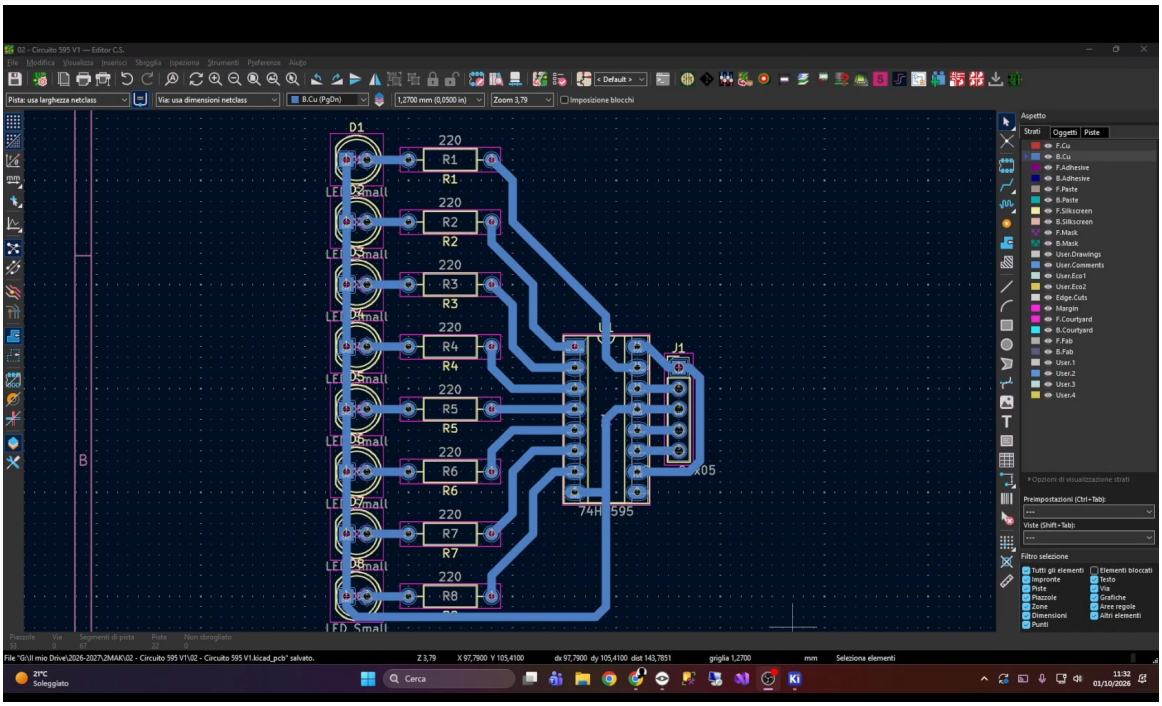


Figura 6: Il circuito sbrogliato al 100% ("Non sbrogliato: 0", in basso a sinistra): tutte le piste tra resistori, LED, il 595 (U1) e il connettore J1 sono tracciate senza angoli a 90 gradi, con gli otto LED allineati verticalmente a sinistra – questo è il risultato finale a cui deve arrivare lo sbroglio.

5 Dal PCB al modello 3D

Definizione

Per definire i **confini fisici** della scheda, va disegnato un rettangolo (o qualunque altra forma) sul layer **Edge Cuts**: senza questo passaggio, l'**esportazione del modello 3D** fallisce, perché KiCad non sa dove si trova il "**bordo di taglio**" della scheda.

Nota

Per **esportare il modello 3D** della scheda (menu File → Esporta → STEP): **preferire sempre** il formato **STEP** rispetto a **STL**. Lo STL è un corpo **unico**, fatto di triangoli, monocromatico: una volta importato in un software di modellazione (es. **Fusion 360**) non si possono più **separare o nascondere** i singoli componenti. Lo STEP, al contrario, esporta ogni componente come un pezzo **separato** (PCB, ogni chip, ogni LED, ecc.): in Fusion si possono **spostare, nascondere o modellare** attorno a ciascun pezzo individualmente. In un circuito come questo, l'importazione dello STEP in Fusion genera circa **40 pezzi distinti** (il PCB, ogni resistore, ogni LED, il chip con il relativo socket, il connettore, ecc.): un numero che rende ancora più evidente perché **raggrupparli rigidamente** sia il **primo passo** da fare.

Attenzione

Dopo aver **importato lo STEP** in Fusion 360, il **primo passo** è creare un **gruppo di relazione rigida** tra **tutti i componenti**: senza questo passaggio, spostando la scheda si rischia di **spo-**

stare un singolo componente (es. un LED) separandolo dal resto, perdendo la **correlazione con il disegno originale**.

Nota

Per **fissare la scheda** dentro una **scatola** si usano dei **mounting hole** (**punti di montaggio**): componenti dedicati che, posizionati sullo schema, definiscono dove praticare i **fori di fissaggio**. Un sistema di fissaggio comune è **a scatto**: la scatola ha un **bordo flessibile** che si piega leggermente per permettere l'inserimento della scheda, poi torna in posizione bloccandola; per rimuoverla basta **fare leva** nel punto giusto. I mounting hole **non sono sempre obbligatori**: in alcuni casi i fori di fissaggio si possono evitare del tutto, a seconda di come la scheda viene poi effettivamente alloggiata.

Esempio

Anticipazione discussa in classe, da sviluppare in una lezione futura: anziché usare un **connettore separato** verso l'Arduino, è possibile inserire il **simbolo dell'Arduino** stesso direttamente nello schema KiCad e progettare uno **shield** – una scheda che si innesta direttamente sopra l'Arduino, **senza fili volanti**. La difficoltà pratica da affrontare sarà la saldatura: in uno shield, i componenti stanno sopra la scheda mentre le piste stanno sotto, quindi il connettore che si innesta sull'Arduino va saldato dal lato opposto a quello dei componenti – un vincolo di montaggio in più rispetto al circuito con connettore separato visto in questa lezione.

6 Cosa ricordare di questa lezione

- Il **74HC595** è pilotato da tre soli segnali effettivi: **SER** (dato seriale), **SRCLK** (fa avanzare il bit nello shift register interno), **RCLK** (pubblica tutti gli 8 bit insieme sulle uscite, evitando lo scorrimento visibile).
- **SRCLR** e **OE** restano sempre fissi (rispettivamente a $+5$ V e GND) nel nostro uso.
- Più 595 si possono collegare in **cascata** tramite il piedino Q'_H, condividendo SRCLK e RCLK: si **moltiplicano le uscite** pilotabili restando sempre a **3 fili** sul microcontrollore.
- In KiCad: assegnare sempre il **footprint** corretto a ogni componente (resistore THT P10,16, LED 5 mm flat top, pin header verticale 2,54 mm, 595 DIP con **socket**).
- **Mai** curve a 90 gradi nello sbroglio delle piste; usare sempre una griglia a 2,54 mm o 1,27 mm, mai in millimetri interi, per restare allineati al passo dei componenti.
- Per il modello 3D: sempre formato **STEP** (componenti separati), mai STL (corpo unico); definire sempre il layer **Edge Cuts** prima di esportare.